Deep Network?

PART 1 — What Actually Happens in a Deep Network?

Let's assume a small but real architecture:

$$2 \to 3 \to 2 \to 1$$

Meaning:

- 2 inputs
- 3 neurons in Hidden Layer 1
- 2 neurons in Hidden Layer 2
- 1 output neuron

Now we explain everything step by step.

Step 1 — Forward Pass with Multiple Neurons

Hidden Layer 1 (3 neurons)

Each neuron receives **2 inputs**.

Each neuron computes a weighted sum + bias.

For neuron jjj in Layer 1:

$$z_j^{(1)} = w_{j1}^{(1)} x_1 + w_{j2}^{(1)} x_2 + b_j^{(1)}$$

Since we have 3 neurons:

$$z_1^{(1)}, \quad z_2^{(1)}, \quad z_3^{(1)}$$

Each neuron has:

- 2 weights
- 1 bias

Total parameters in Layer 1:

$$3 \times (2 + 1) = 9$$

After that, apply activation:

$$a_j^{(1)} = f(z_j^{(1)})$$

Now we have:

$$a_1^{(1)}, \quad a_2^{(1)}, \quad a_3^{(1)}$$

These become inputs to next layer.

Hidden Layer 2 (2 neurons)

Each neuron now receives **3 inputs** (from previous layer).

For neuron k in Layer 2:

$$z_k^{(2)} = w_{k1}^{(2)} a_1^{(1)} + w_{k2}^{(2)} a_2^{(1)} + w_{k3}^{(2)} a_3^{(1)} + b_k^{(2)}$$

Each neuron has:

- 3 weights
- 1 bias

Total parameters in Layer 2:

$$2 \times (3 + 1) = 8$$

After activation:

$$a_k^{(2)} = f(z_k^{(2)})$$

Now we have:

$$a_1^{(2)}, \quad a_2^{(2)}$$

Output Layer (1 neuron)

Receives 2 inputs:

$$z^{(3)} = w_1^{(3)} a_1^{(2)} + w_2^{(3)} a_2^{(2)} + b^{(3)}$$

Parameters in output layer:

$$2 + 1 = 3$$

Final output:

$$\hat{y} = f(z^{(3)})$$

Total Parameters

Layer 1:

$$9$$

Layer 2:

$$8$$

Layer 3:

$$3$$

Total:

$$9 + 8 + 3 = 20$$

- Each layer is fully connected to the next.
- Each neuron has its own bias.

- Bias count does NOT multiply across iterations.
- Bias belongs to the neuron, not to each input.

🔹 Step 2 — What Is Activation Slope?

Activation slope means:

$$f'(z)$$

It is the derivative of the activation function.

It tells:

If input changes slightly, how much does output change?

Example 1 — Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Derivative:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Maximum value:

$$0.25$$

So slope is always ≤ 0.25.

Meaning:

Gradients shrink at every layer.

Example 2 — ReLU

$$f(z) = \max(0, z)$$

Derivative:

$$f'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

So slope is either 1 or 0.

Important:

Slope is NOT bias.

Bias shifts curve left or right.

Slope determines:

How strongly output changes when input changes.

During backpropagation, slope multiplies gradient.

This is critical.

Step 3 — What Happens in Backpropagation?

Assume loss:

$$L = \frac{1}{2}(y - \hat{y})^2$$

We compute gradients layer by layer.

First:

$$\frac{\partial L}{\partial z^{(3)}}$$

Then:

$$\frac{\partial L}{\partial a^{(2)}}$$

Then:

$$\frac{\partial L}{\partial z^{(2)}}$$

Then:

$$\frac{\partial L}{\partial a^{(1)}}$$

Then:

$$\frac{\partial L}{\partial z^{(1)}}$$

At every layer:

$$\delta^{(l)} = \text{incoming gradient} \times f'(z^{(l)})$$

So gradient at each layer is multiplied by:

- weights
- activation slope

This is the key.

Step 4 — Do We Update Layer by Layer?

Important clarity:

We do NOT update one layer and immediately move forward.

Correct sequence:

1. Forward pass through ALL layers
2. Compute loss
3. Backward pass through ALL layers
4. Compute gradients for ALL weights
5. Update ALL weights together

We do not partially update during backward traversal.

Because gradients must be computed based on SAME forward state.

If you update early, later gradients become inconsistent.

Step 5 — Why Gradients Multiply

Suppose activation slope is kkk.

Across layers:

$$\delta^{(1)} = \delta^{(3)} \times k_2 \times k_1$$

If k<1k < 1k<1 repeatedly:

Gradient shrinks exponentially.

If k>1k > 1k>1 repeatedly:

Gradient explodes.

This is independent of number of neurons.

More neurons does NOT fix vanishing.

Depth causes multiplication chain.

Step 6 — What About Bias During Backprop?

Bias gradient is simple:

$$\frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}$$

Bias does not multiply with inputs.

It just shifts output.

It does not grow with iterations.

It has one gradient per neuron.

Bias count stays separate because:

Each neuron has one bias.

Not one bias per input.

Step 7 — If We Had More Neurons?

Compare:

Case A:

$$2 \to 3 \to 2 \to 1$$

Case B:

$$2 \to 6 \to 4 \to 1$$

More neurons increase width.

Parameter count increases.

But gradient chain depth is same (3 layers).

Vanishing depends on depth, not width.

Final Conceptual Summary

Before DLQ6, must clearly understand:

- What is a neuron?
- What is a layer?
- How parameters are counted?
- What is activation slope?
- How gradient flows backward?
- Why gradients multiply?
- Why depth causes vanishing?
- Why width does not fix it?
- Why updates happen after full backward pass?